Department of Computer Science and Engineering

CAPSTONE PROJECT REPORT

Embedded Systems

Multi-Node BLE-Based Classroom Presence Verification System

Kavin J S (CH.SC.U4CSE24119)

Faculty guide: Dr. J. V. Aravind

October 2026

1. Introduction to the Topic

Calling out names for attendance takes time, mistakes happen, and students can get a friend to answer for them. Shaikh et al., the main reference paper for this report, talk about this problem, and GateSense says roll call can use up about 10 minutes of every lecture hour. Another problem is with systems that check a student in only once. A student can be counted when class starts, walk out a few minutes later, and still be shown as present.

Our idea is simple. If a student skips or leaves in the middle of a class, they should be marked absent. So we do not check just once. The system keeps checking for the whole class, and the final result comes only when the teacher ends the class.

We use two ESP32 boards that listen for Bluetooth Low Energy (BLE) signals from each student's registered phone. Both boards send what they hear to a Flask server on a laptop over Wi-Fi. Every 10 seconds the server checks each student, saves the result in a CSV file and shows live status on a web dashboard. A student only has to open the free nRF Connect app and advertise a fixed name. In our demo we registered two phones, one of them named ATTEND-KAVIN.

We set four objectives. First, detect registered phones using two scanning nodes. Second, count a student as present only when both nodes agree. Third, mark a student absent if they leave in the middle of class. Fourth, keep the build cheap and easy to explain, with no machine learning and no custom student app. The system can tell that a registered phone is in the room, but it cannot tell who is carrying it. We save RSSI in the log for reference and never turn it into a distance.

2. Literature Related to the Work

Shaikh, Gupta, Pal and Singh (IJERT, Vol. 15, Issue 04, April 2026) made a Bluetooth attendance system that runs on smartphones. The teacher's phone sends out a session ID over BLE. Each student's phone picks it up and sends it to a backend. Attendance is accepted if the session ID matches, the RSSI shows the student is close enough, and a face check confirms who they are. Their app is built with React Native, the backend uses Node.js and Express, and the data is stored in PostgreSQL through Prisma. They also compared seven attendance methods on hardware, security and scalability, and gave teacher-broadcast BLE a high rating for both security and scalability.

They tested a BLE range of 10 to 20 m, indoor interference, many students scanning together, and different lighting for the face check. Their conclusion is that BLE with face authentication reduces proxy attendance a lot, but they do not give any accuracy numbers. They also list their own problems: RSSI is unstable near walls and crowds, phone models behave differently, batteries drain faster, and internet is required. For future work they suggest AI/ML-based RSSI analysis, BLE combined with Wi-Fi or GPS, and iOS support. We take their BLE idea, the server-side checking, the dashboard and their warning about RSSI. What we change is the sensing side. We use fixed ESP32 scanners in the room, a free advertiser app on the student phone, and one decision made over the whole class.

GateSense (Journal of Applied Science, Engineering, Technology and Management, 2026) points out the same gap. Normal methods waste about 10 minutes of lecture per hour, and they

cannot confirm that a student is still present after the first check-in. It uses an ESP32 with a fingerprint sensor, two IR sensors and an RFID reader, and reports 98% directional detection accuracy over 100 trials with 1.5 s average cloud latency. It keeps track of students using hardware at the door. Vieira et al. (SBCUP 2026) use ESP32 scanners with ESPresense, MQTT, Home Assistant and a Python middleware. In three simulated 60-minute runs with 30 synthetic students, only 14.38% of the 21,600 messages per run became stored events, and stay-phase attendance averaged 27.43 of 30 students (91.43%). Their sensing is the closest to ours, but it needs several services running together.

Flueratoru et al. (IEEE Sensors Journal, 2022) measured BLE signal strength at two universities. They found that it varies a lot in ways that standard path-loss models cannot explain, and it varies even more when the three advertising channels are combined. This supports the RSSI warning from Shaikh et al. It is also the reason we decide presence by how often a phone is heard, not by how strong the signal is.

To sum up, the main reference paper decides attendance once, using a session ID, an RSSI range and a face match, and gives no accuracy numbers. GateSense depends on hardware at the door and reports 98% direction accuracy over 100 trials. The middleware of Vieira et al. filters room-level presence and reports 91.43% stay-phase attendance. Our work uses two ESP32 scanners that have to agree, checks every 10 s, and applies a class rule of 75% in range with no gap over 5 minutes. We tested it on nine simulated scenarios and on the real campus Wi-Fi.

3. Methodology and Framework Implemented

Our first plan was three ESP32 nodes with adaptive thresholds and majority voting. The capstone guidebook cut this down to two nodes with rule-based checking, since majority voting makes no sense with only two nodes. In our first working version, a student was marked present as soon as both nodes had heard the phone 3 times. We later replaced that with a rule over the whole class, so that leaving in the middle gives ABSENT.

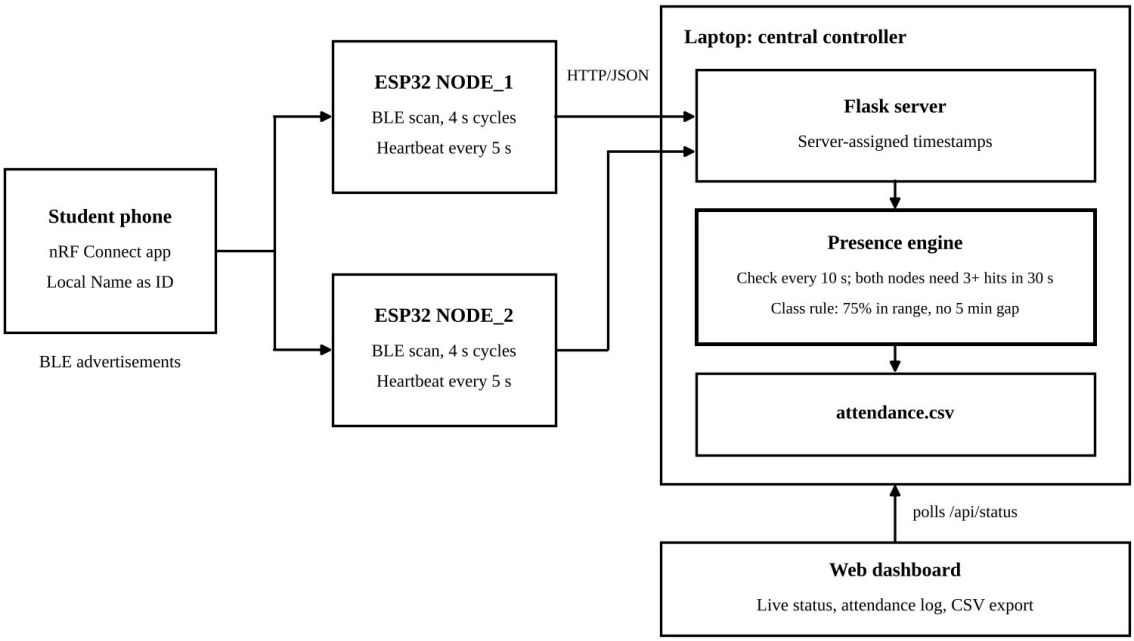


Figure 1: System architecture

Each phone advertises a fixed BLE Local Name. Both nodes scan for 4 seconds at a time, compare the names they hear with the registered list, and send JSON to the laptop over HTTP. Each sighting has the node id, the device name and the RSSI. Each heartbeat has the node id, status, uptime and free memory, and is sent every 5 seconds before a scan. The server adds its own timestamp to every record, so the boards do not need their clocks synchronised.

The hardware is two ESP32-WROOM-32 38-pin boards (INR 428 to 499 each, September 2026 listings), two USB-C data cables (about INR 150 to 300 in total) and an optional LED kit with eight LEDs, resistors, a breadboard and jumpers (about INR 250). The two power banks and the laptop were already with us.

With cables, the core hardware costs about INR 1,350 to 1,700. The ESP32 supports 802.11 b/g/n Wi-Fi on 2.4 GHz and Bluetooth v4.2 BR/EDR and BLE. When we scanned the campus network from the laptop, we saw access points on channels 1, 13, 40 and 48. The ESP32 cannot use 5 GHz, so we fixed each node to a 2.4 GHz access point by BSSID and channel and turned Wi-Fi modem sleep off. The HTTP timeout is 8 seconds. Four LEDs on GPIO 25, 26, 27 and 14 show Wi-Fi ready (green), scanning (blue), a successful send (yellow flash) and a fault (red).

A phone counts as in range at time t only when each node has heard it at least 3 times in the last 30 seconds, that is N1(t) >= 3 and N2(t) >= 3. Asking both nodes is stricter than trusting one. Every 10 seconds the server runs this test for each student. If a node's heartbeat is stale (about 45 seconds), that check is skipped for everyone, so a network fault is not held against students. When the teacher ends the class, a student is PRESENT only if the ratio R of passed checks to valid checks is at least 0.75 and the longest out-of-range gap G is at most 5 minutes. If not, the student is ABSENT and the reason is saved. One row per student per class goes into data/attendance.csv, and a snapshot lets a restarted server carry on with the class.

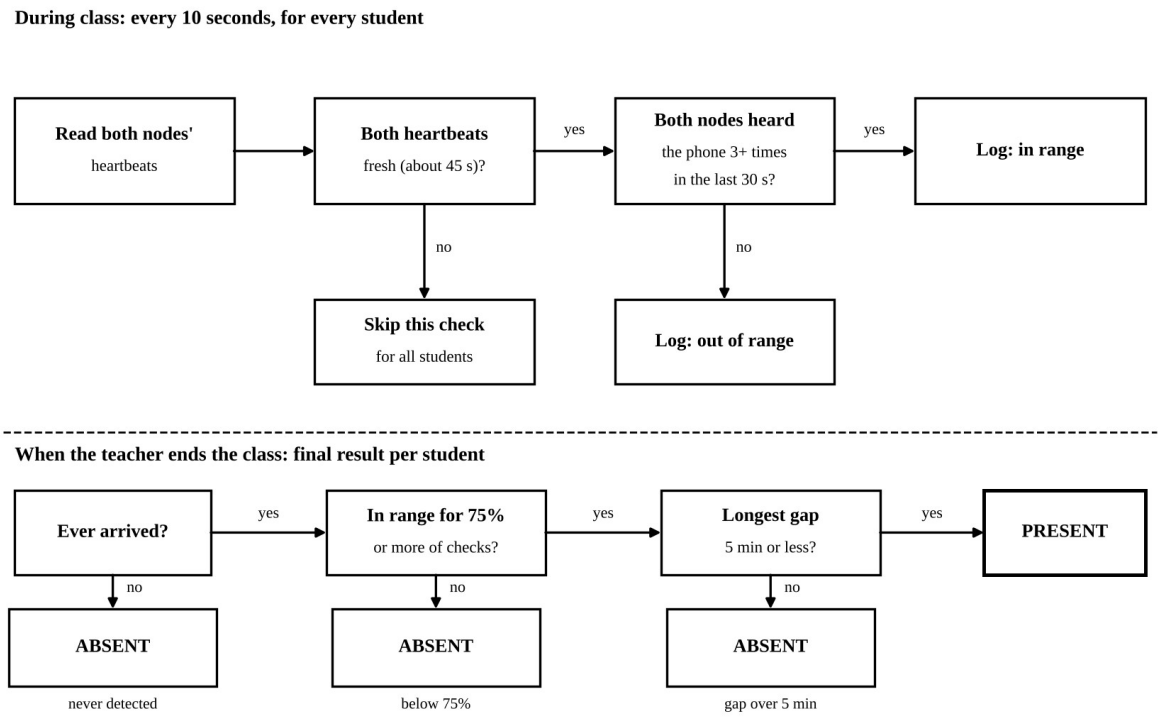


Figure 2: Decision flow during class and at the end of class

The values we settled on are a 30 s presence window, 3 detections needed per node and a 10 s check interval. The firmware uses a 5 s heartbeat and an 8 s HTTP timeout, and the server treats a heartbeat older than about 45 s as stale. We chose the last two after measuring the network delays. For the attendance policy, a student needs at least 75% of checks in range and no gap longer than 5 minutes, and a course can change both.

All of these values are constants at the top of server.py and the node files. The 75% value comes from the usual Indian university attendance rule (Section 7). The 5-minute gap is our own choice and we have not measured it. The Flask server has endpoints for detections, heartbeats, live status, attendance history, an event feed, RSSI history, CSV export and class control. The dashboard reads from them and shows node health, which node hears which phone, a timeline for each student with an "if the class ended now" verdict, an activity feed and the saved log. It works offline, with no external scripts or fonts.

4. How the Present Work Differs from Earlier Work

Aspect	Main reference paper	This work
Who scans	Student phones scan for the teacher's broadcast	Two fixed ESP32 boards scan for student phones
Student-side software	React Native app on every phone, plus camera	Free nRF Connect advertiser, no custom app
When attendance is decided	Once, when session ID, RSSI range and face match pass	Every 10 s, with a final result when the class ends
Leaving mid-class	Not addressed after the request is accepted	Gap over 5 min or presence under 75% gives ABSENT
Identity	Face recognition (needs camera, light, internet)	Device presence only, no biometrics collected
Backend	Node.js, Express, PostgreSQL, Prisma	One Flask process and a CSV file
Sensor agreement	One student phone's view	Both nodes must agree
Network failure	Internet needed for verification	Checks skipped when a node goes silent, local network is enough
Reported evidence	Qualitative	Simulation and network measurements (Section 5)

Compared with GateSense, our work does not need any entry or exit hardware. Compared with the middleware of Vieira et al., it does not need an MQTT broker or Home Assistant. What we lose is the identity check that Shaikh et al. and GateSense get from face or fingerprint sensing.

5. Results and Discussion

We collected three kinds of evidence. First, we ran simulated classes through the real server on a fake clock, which let us test a 60-minute class in a few seconds. Second, we ran the two nodes on the campus Wi-Fi with registered phones and watched the serial output, the dashboard and the CSV. Third, we measured the network with ping from the laptop. We have not measured accuracy in a full real classroom. Each simulated class was 20 minutes long unless a length is given, and all nine scenarios gave the result we expected.

A student who stays for the whole class is marked PRESENT with 99% of checks in range. A student who walks out after 5 minutes is shown live as "Left the room" and ends up ABSENT. A 3-minute break and a 4-minute gap in a 60-minute class both give PRESENT, but a 6-minute gap in a 60-minute class gives ABSENT. A late student who misses 8 of 60 minutes is PRESENT with 86% in range. A student who is never detected is ABSENT with the reason "never detected in class". If Node 2 is down for 10 minutes, the student stays PRESENT because those checks are skipped. If the server restarts in the middle of a class, the class carries on from the snapshot.

Both mid-class exits were handled correctly, which a check-in-only scheme like the one in the main reference paper cannot do. The gap tests sit on both sides of the 5-minute limit, and the late-arrival case shows that a student who comes late but stays can still pass the 75% rule.

On the campus network, the laptop and the ESP32 were in the same /19 subnet (11.12.0.0 to 11.12.31.255), so our first guess that the network blocks devices from talking to each other was wrong. Ping round-trip times went from 20 ms up to 6,607 ms, with 10% to 40% packet loss and some duplicate packets, and the original 3-second HTTP timeout caused code -1 failures on heartbeats. We raised the timeout to 8 s, raised the server's stale-heartbeat limit from 20 s to about 45 s, and fixed the node to one 2.4 GHz access point.

We found and fixed four defects. First, the node identifier in node_2 was written "Node_2" but the server expects "NODE_2", so the server rejected 100% of Node 2 requests with HTTP 400 and the both-node check could never pass. After a one-line change the requests returned HTTP 200 and the student was verified after 3 detections per node. Second, the CSV path depended on the folder the terminal was in, so we tied it to the script location. Third, the check for duplicate attendance lived only in memory, so a restart could add a second row. The state is now rebuilt from the CSV and a snapshot, and a simulated restart left exactly 1 row. Fourth, attendance stayed recorded even after a student left, and this is why we replaced it with the class-long rule.

With the registered phones advertising, the nodes posted heartbeats and detections, and the dashboard showed node status, per-node counts and RSSI. When Bluetooth was switched off on a phone, both counts dropped to 0 of 3 within the 30-second window. There are limits we should be clear about. The scenarios test the rule logic, not how the radio behaves in a full room. A student who steps out shows as out of range after about 30 seconds and as having left after 5 minutes, and the gap can only be measured to about 10 seconds. A phone that locks and stops advertising looks like the student left. A shared phone would still be counted, which is the same weakness as RSSI proximity in the main reference paper. The 30 s, 3-detection, 75% and 5-minute values are starting points that need tuning, and we tested with only two registered phones.

6. Novelty and Improvements with Quantification

Every figure here comes from the tests in Section 5 or follows from the stated parameters, and derived figures are marked.

Improvement	Before	After	Basis
Node 2 requests accepted by the server	0%	100%	Measured, HTTP 400 to 200
Mid-class exits marked ABSENT	Not possible in the first version	2 of 2 correct	Simulated, 5-minute walk-out and 6-minute gap
Time resolution of presence	One decision per class	One check per 10 s, 360 checks in 60 min	Derived from the check interval
Pass condition on a 60-minute class	None	At least 270 passed checks and no gap over 30 consecutive checks	Derived, 360 x 0.75 = 270 and 300 s / 10 s = 30
HTTP timeout against worst round trip	3 s against 6.6 s	8 s against 6.6 s	Measured round trip, firmware constant
Stale-heartbeat limit	20 s	about 45 s	Server constant, set above the 5 s heartbeat
Checks wrongly charged in a node outage	Up to 60 s	At most 30 s	Simulated, Node 2 down 10 min left the student PRESENT
Duplicate rows after a server restart	2 rows possible	1 row	Simulated restart and re-detection
Sensing hardware cost	Not applicable	INR 856 to 998 for two boards	Listed prices, INR 428 to 499 each
Software on each student phone	Custom app plus camera in Shaikh et al.	Free nRF Connect advertiser only	Design comparison with the main reference paper

What is new here is a combination of four things. The first is two-node agreement as the presence test. The second is a class-long rule that uses both a time-in-range ratio and a maximum gap. The third is checking that pauses when a node goes down. The fourth is a build that needs one laptop, two boards under INR 1,000 and no student app. None of the three papers we reviewed combines all of these.

7. Importance to Society and the Nation

Most Indian universities ask for at least 75% attendance before a student can sit the exams. This comes from UGC regulations and is applied through university ordinances. A system that measures the time a student really spends in class is fairer for this rule than one roll call. If roll call takes about 10 minutes per hour, as GateSense reports, automating it gives up to one sixth of every lecture back to teaching. We took that figure from GateSense and did not measure it ourselves.

The sensing hardware costs about INR 1,000, so colleges that cannot pay for RFID, fingerprint or face-recognition setups can still use it, and the records stay on a local laptop. We collect no photos or biometrics, only a chosen device name and signal strength. This is less intrusive than the face-verification layer in Shaikh et al. and fits the data-minimisation principle in India's Digital Personal Data Protection Act, 2023. Since checks are skipped during a node outage, a student is not marked absent just because the Wi-Fi failed, and the saved reason lets a teacher review any ABSENT result.

8. Suggestions for Future Work

- Run a real-classroom trial with 30 or more students and record detection, false-absent and false-present rates against a manual roll call.
- Tune the window, detection count, 75% and 5-minute values by experiment, using the logged RSSI and detection counts.
- Add an identity layer such as a one-time code or rotating BLE payload per student, to address the shared-phone limit, in the spirit of the multi-factor idea in the main reference paper.
- Add a third node, at which point majority voting becomes meaningful.
- Build a small phone app that keeps advertising in the background, since the free advertiser must stay in the foreground on some Android versions.
- Use smoothed RSSI (for example a Kalman filter) as a secondary check only after calibration in the real room, because single readings are unreliable, as Flueratoru et al. show.
- Move to MQTT or a database when several classrooms share one controller, and export to the college ERP in place of the CSV.

9. Conclusion

This project takes the BLE proximity attendance idea from the main reference paper and extends it from a one-time check to a decision over the whole class. Two ESP32 scanners, a Flask server and a free phone app were enough to detect registered phones, make both nodes agree, and mark a student ABSENT after leaving in the middle of class, while staying steady through short breaks and network problems. All nine simulated scenarios gave the expected result, and on the campus network we measured round-trip times of up to 6.6 seconds and the system handled them. The main limits are that it identifies devices and not people, and that we have not yet tried it with a full class. These are the next steps.

10. References

S. Shaikh, S. Gupta, S. Pal and A. Singh (guide: K. Sheikh), "Bluetooth-Based Smart Attendance System using BLE and Mobile Devices," International Journal of Engineering Research & Technology (IJERT), vol. 15, no. 4, pp. 1-5, Apr. 2026, ISSN 2278-0181, paper ID IJERTV15IS040317. (Main reference paper)

N. K. Abhirami, K. C. Amidha, A. M. George, A. P. S. Sharaf and L. Thomas, "GateSense: A Smart Continuous Attendance System," Journal of Applied Science, Engineering, Technology and

Management, vol. 4, no. 1, pp. 35-41, 2026. doi: 10.61779/jasetm.v4i1.7. https://jasetm.fisat.ac.in/index.php/jasetm/article/view/63

E. de F. Vieira, T. A. Rizzetti and R. P. De Azevedo, "Automated Classroom Attendance in Educational Environments Using a BLE-Based Context-Aware Middleware," in Anais do XVIII Simposio Brasileiro de Computacao Ubiqua e Pervasiva (SBCUP), Gramado, Brazil, pp. 81-91, 2026. doi: 10.5753/sbcup.2026.23393. https://sol.sbc.org.br/index.php/sbcup/article/view/43289

L. Flueratoru, V. Shubina, D. Niculescu and E.-S. Lohan, "On the High Fluctuations of Received Signal Strength Measurements With BLE Signals for Contact Tracing and Proximity Detection," IEEE Sensors Journal, vol. 22, no. 6, Mar. 2022. https://trepo.tuni.fi/handle/10024/220480

11. What We Learnt from This Capstone

This capstone showed us that an embedded system often fails because of small details between its parts. One capital letter in a node identifier made the server reject every message from one board. It looked like a radio problem until we compared the node's code with the server's checks. We also learnt to measure before blaming. A ping test showed the campus network was congested and not blocked, so our fix was longer timeouts and not a different network. One requirement also changed the whole design. Once we decided that a student who leaves in the middle must be absent, we had to replace the one-time record with repeated checks and a final rule. Along the way we learnt to test logic on a simulated clock, to keep timestamps on one trusted machine, to treat a silent node differently from a failed message, and to say clearly what the system proves, which is that a device is in the room and not that the right person is carrying it. Writing this report also taught us to read a reference paper critically, using its strengths and listing its limits, and to report only the numbers we actually measured.